

In summary, we described a reduction of the problem of checking whether a candidate c is a possible winner to an ILP, and proposed a three-phase computation that limits the number of times the ILP solver is invoked for a set of candidates C . We will show experimentally in Section 6.4 that the proposed techniques can be used to compute the set of possible winners in realistic scenarios.

5 The Repeated Selection Model for Poset Generation

In this section we introduce a novel generative model for partially ordered sets, called the Repeated Selection Model, or RSM for short. It includes earlier generative models of partial orders as special cases via a suitable choice of parameters. We regard RSM as being a model of independent interest, and we also use it here as part of our experimental evaluation, described in Section 6. To start, we introduce the Repeated Insertion Model (RIM) that is used for generating total orders in Section 5.1. We then describe our novel RSM model in Section 5.2.

5.1 Preliminaries: The Repeated Insertion Model (RIM)

In this section we represent total orders using rankings, that is, ordered lists of items indexed by position. We will use σ , τ , and so on to denote rankings. We will use $\sigma(i)$ to refer to an item at position i in σ , and we will use $\sigma^{-1}(a)$ to denote the position of element a in σ . When describing iterative algorithms, for convenience of presentation we will denote by σ_i the value of σ at step i .

The Repeated Insertion Model (RIM) is a generative model that defines a probability distribution over rankings due to Doignon et al. [14]. This distribution, denoted by $\text{RIM}(\sigma, \Pi)$, is parameterized by a reference ranking σ and a function Π , where $\Pi(i, j)$ is the probability of inserting $\sigma(i)$ at position j . Here, Π is a matrix where each row corresponds to a valid probability distribution (i.e., the values in a row sum up to one). Algorithm 2 presents the RIM sampling procedure. It starts with an empty ranking τ , inserts items in the order of σ , and puts item $\sigma(i)$ at j^{th} position of the currently incomplete τ with probability $\Pi(i, j)$. The algorithm terminates after m iterations, and outputs τ , a total order over the items drawn from σ .

Algorithm 2 $\text{RIM}(\sigma, \Pi)$

```

1: Initialize an empty ranking  $\tau = \langle \rangle$ .
2: for  $i = 1, \dots, m$  do
3:   Select a random position  $j \in [1, i]$  with a probability  $\Pi(i, j)$ 
4:   Insert  $\sigma(i)$  into  $\tau$  at position  $j$ 
5: end for
6: return  $\tau$ 

```

Example 1. $\text{RIM}(\langle a, b, c \rangle, \Pi)$ generates $\tau = \langle b, c, a \rangle$ as follows.

- Initialize an empty ranking $\tau_0 = \langle \rangle$.
- At step 1, $\tau_1 = \langle a \rangle$ by inserting a into τ_0 with probability $\Pi(1, 1) = 1$.
- At step 2, $\tau_2 = \langle b, a \rangle$ by inserting b into τ_1 at position 1 with probability $\Pi(2, 1)$. Note that b is put before a since $b \succ_{\tau} a$.
- At step 3, $\tau = \langle b, c, a \rangle$ by inserting c into τ_2 at position 2 with probability $\Pi(3, 2)$.

The overall probability of sampling τ is $\Pr(\tau \mid \langle a, b, c \rangle, \Pi) = \Pi(1, 1) \cdot \Pi(2, 1) \cdot \Pi(3, 2)$. Note that this particular sequence of steps is the only way to sample $\langle b, c, a \rangle$ from $\text{RIM}(\langle a, b, c \rangle, \Pi)$.

The Mallows model [15], $\text{MAL}(\sigma, \phi)$, $\phi \in (0, 1]$, is a special case of RIM. As a popular preference model, it defines a distribution of rankings that is analogous to the Gaussian distribution: the ranking σ is at the center, and rankings closer to σ have higher probabilities. Specifically, the probability of a ranking τ is given by:

$$\Pr(\tau \mid \text{MAL}(\sigma, \phi)) = \frac{\phi^{\text{dist}(\sigma, \tau)}}{1 \cdot (1 + \phi) \cdot (1 + \phi + \phi^2) \dots (1 + \dots + \phi^{m-1})} \quad (11)$$

Here, $\text{dist}(\sigma, \tau)$ is the Kendall-tau distance between σ and τ : $\text{dist}(\sigma, \tau) = |(a, a') \mid a \succ_{\sigma} a', a' \succ_{\tau} a|$, that is the number of preference pairs (a, a') that appear in the opposite relative order in σ and τ . The expression in the

denominator of Equation 11 is the normalization constant, which we will find convenient to denote $Z_{\phi, m}$. When $\phi \rightarrow 0$, the probability mass is concentrated around the reference ranking σ ; when $\phi = 1$, all rankings have the same probability, that is, $\text{MAL}(\sigma, 1)$ is the uniform distribution over rankings.

As was shown in [14], $\text{RIM}(\sigma, \Pi)$ is precisely $\text{MAL}(\sigma, \phi)$ when $\Pi(i, j) = \frac{\phi^{i-j}}{1+\phi+\dots+\phi^{t-1}}$. That is, the Mallows model is a special case of RIM, and so RIM can be used as an efficient sampler for Mallows.

5.2 The Repeated Selection Model (RSM)

The Repeated Selection Model (RSM) is a generative model that defines a probability distribution over posets. Intuitively, in this model we iteratively select a random item and randomly choose whether it succeeds each of the remaining items. More formally, an instance of this distribution, denoted $\text{RSM}(\sigma, \Pi, p)$, is parameterized by a reference ranking σ of length m , a selection probability function Π , where $\Pi(i, j)$ is the probability of *selecting* the j^{th} item among the remaining items at step i , and a preference probability function $p : \{1, \dots, m-1\} \rightarrow [0, 1]$ that determines the probability $p(i)$ that the i^{th} selected item precedes (is preferred to) each of the remaining items. We view Π as a matrix where each row corresponds to a valid probability distribution (i.e., the values in a row sum up to one) and the $i-1$ rightmost entries in the i^{th} row are zero.

Algorithm 3 presents the RSM sampling procedure. Intuitively, in contrast to RIM (Algorithm 2) that considers candidates one by one in the order of σ and inserts them into the output τ , RSM iteratively selects, and removes, candidates one by one from among the remaining candidates in σ at each step. Which candidate is selected at step i is decided randomly, based on the probability distribution in the i^{th} row of the selection probability matrix Π (line 4 of Algorithm 3). Furthermore, to generate posets rather than total orders, RSM uses the preference probability function p to decide whether to add a particular preference pair to τ ; this decision is made independently for all considered pairs (lines 6-8 of Algorithm 3). The probability that a candidate selected at step i is preferred to each of the remaining candidates in σ is $p(i)$.

Algorithm 3 $\text{RSM}(\sigma, p, \Pi)$

```

1: Initialize an empty poset  $\tau = \langle \rangle$ .
2: for  $i = 1, \dots, m-1$  do
3:   Select a random position  $j \in [1, m]$  with a probability  $\Pi(i, j)$ 
4:   Select candidate  $c = \sigma(j)$ 
5:   Remove  $c$  from  $\sigma$ , which now contains  $m-i$  candidates
6:   for  $k = 1, \dots, m-i$  do
7:     Add the pair  $c \succ \sigma(k)$  to  $\tau$  with probability  $p(i)$  (or leave it out with probability  $1-p(i)$ )
8:   end for
9: end for
10: return the transitive closure of  $\tau$ 

```

Example 2. $\text{RSM}(\langle a, b, c \rangle, \Pi, p)$ can generate $\tau = \langle b \succ a, a \succ c, b \succ c \rangle$ as follows:

- Initialize an empty poset $\tau = \langle \rangle$.
- At step $i = 1$, select b with probability $\Pi(1, 2)$ and remove it from σ , setting $\sigma_1 = \langle a, c \rangle$. Then, add the pair $b \succ a$ to τ with probability $p(1)$, and *do not add* the pair $b \succ c$ to τ with probability $1-p(1)$.
- At step $i = 2$, select a with probability $\Pi(2, 1)$ and remove it from σ , setting $\sigma_2 = \langle c \rangle$. Finally, add the pair $a \succ c$ to τ with probability $p(2)$.
- Take the transitive closure of τ and return $\langle b \succ a, a \succ c, b \succ c \rangle$.

The probability of sampling τ in this way is $\Pi(1, 2) \cdot p(1) \cdot (1-p(1)) \cdot \Pi(2, 1) \cdot p(2)$.

Note that the same τ can be generated by $\text{RSM}(\langle a, b, c \rangle, \Pi, p)$ using a different sequence of steps, thus yielding a different probability. In our example there is one other way to derive τ : at step $i = 1$, add $b \succ c$ to τ with probability $p(1)$. This happens with the probability $\Pi(1, 2) \cdot p(1) \cdot p(1) \cdot \Pi(2, 1) \cdot p(2)$. These are the only two possible derivations of τ in our example. These yield the total probability

$$\Pr(b \succ a \succ c \mid \langle a, b, c \rangle, \Pi, p) = \Pi(1, 2) \cdot p(1) \cdot \Pi(2, 1) \cdot p(2).$$

In the general case, however, it is not clear whether this probability can be computed efficiently. In particular, the probability of a poset may be due to all the linear extensions of the poset.

Importantly, RSM includes several generative models of partial orders as special cases via a suitable choice of parameters. For example, $p = (1, \dots, 1, 0, \dots, 0)$ with k ones will generate a top-truncated partial order, whereas $p = (0, \dots, 0, 1, \dots, 1)$ with k ones will generate a partial chain over a subset of k items. Moreover, a uniform p gives rise to the generative model referred to as *Method 1* of Gehrlein [16]. Figure 1 compares these probability distributions empirically. Finally, as we show below, the Mallows model is also a special case of RSM.

Theorem 3. *For a given $\phi \in (0, 1]$, and for $p(i) = 1$ for all i , we have that $\text{RSM}(\sigma, \Pi, p)$ is precisely $\text{MAL}(\sigma, \phi)$ when $\Pi(i, j) = \frac{\phi^{j-1}}{\sum_{k=1}^{m-i+1} \phi^{k-1}}$.*

Proof. First, observe that because $p(i) = 1$ for all i , $\text{RSM}(\sigma, \Pi, p)$ will generate total orders. Further, observe that, although an arbitrary poset can be generated by RSM in multiple ways (as demonstrated by Example 2), there is only one way to obtain a total order (ranking). We will show by induction on the number of candidates m in σ that $\Pr(\tau \mid \text{RSM}(\sigma, \Pi, p)) = \Pr(\tau \mid \text{MAL}(\sigma, \phi))$. Recall from Equation 11 that $\Pr(\tau \mid \text{MAL}(\sigma, \phi)) = \phi^{\text{dist}(\sigma, \tau)} / Z_{\phi, m}$, where $Z_{\phi, m}$ is a normalization constant, and $\text{dist}(\sigma, \tau)$ is the Kendall-tau distance between σ and τ : $\text{dist}(\sigma, \tau) = |(a, a') \mid a \succ_{\sigma} a', a' \succ_{\tau} a|$, that is the number of preference pairs (a, a') that appear in the opposite relative order in σ and τ . For notational convenience, we will denote by σ_{-a} a subranking of σ with item a removed. Further, we will denote by $\Pi_{-i, -j}$ a projection of the matrix Π with the i^{th} column and j^{th} row removed.

Base case. When $m = 1$, both RSM and Mallows generate a single ranking with probability 1.

Inductive step. Suppose that RSM and Mallows assign the same probability to the subranking of some τ with the first element $\tau(1)$, denoted τ_1 , removed, and with σ and Π adjusted accordingly:

$$\Pr(\tau_{-\tau_1} \mid \text{RSM}(\sigma_{-\tau_1}, \Pi_{-1, -m}, p)) = \Pr(\tau_{-\tau_1} \mid \text{MAL}(\sigma_{-\tau_1}, \phi)) = \frac{\phi^{\text{dist}(\sigma_{-\tau_1}, \tau_{-\tau_1})}}{Z_{\phi, m-1}} \quad (12)$$

Let us now consider the ranking τ of length m , and observe that $\text{dist}(\tau, \sigma) = \text{dist}(\tau_{-\tau_1}, \sigma_{-\tau_1}) + \sigma^{-1}(\tau_1) - 1$, where $\sigma^{-1}(\tau_1)$ is the position of element τ_1 in σ . Moreover, the probability to select τ_1 at the first step of RSM is given by:

$$\Pi(1, \sigma^{-1}(\tau_1)) = \frac{\phi^{\sigma^{-1}(\tau_1)-1}}{\sum_{k=1}^m \phi^{k-1}} \quad (13)$$

Combining Equations 12 and 13, and recalling the expression for $Z_{\phi, m}$ from Equation 11, we obtain the following probability for τ :

$$\begin{aligned} \Pr(\tau \mid \text{RSM}(\sigma, \Pi, p)) &= \Pi(1, \sigma^{-1}(\tau_1)) \times \Pr(\tau_{-\tau_1} \mid \text{RSM}(\sigma_{-\tau_1}, \Pi_{-1, -m}, p)) \\ &= \frac{\phi^{\sigma^{-1}(\tau_1)-1}}{\sum_{k=1}^m \phi^{k-1}} \times \frac{\phi^{\text{dist}(\sigma_{-\tau_1}, \tau_{-\tau_1})}}{Z_{\phi, m-1}} = \frac{\phi^{\text{dist}(\tau, \sigma)}}{Z_{\phi, m}} = \Pr(\tau \mid \text{MAL}(\sigma, \phi)). \end{aligned}$$

The proof by induction concludes and the theorem is proven. $\square$

6 Experimental Evaluation

All experiments were carried out on an Intel(R) Xeon(R) CPU E5-2680 v3 @ 2.50GHz, with 412 GB of RAM, 20 hyper-threaded cores running 2 threads per core, running Ubuntu 16.04.6 LTS. We used Python 3.5 for our implementation, and the solver Gurobi v8.1.1 [17] for solving the ILP instances produced by the reduction from instances of the possible winner problem.

6.1 Experimental Datasets and Scoring Rules

Real datasets We used two real datasets in our experimental evaluation, *travel* and *dessert*.

The Google Travel Review Ratings dataset (*travel*) [18] consists of average ratings (each between 1 and 5) issued by 5,456 users for up to 24 travel categories in Europe. For each user, we create a set of preference pairs such that items in each pair have different ratings (no tied pairs). Items for which a user does not provide a rating are not included into that user's preferences. Because preferences are derived from ratings issued by individual users, there cannot be any cycles in the set of preference pairs corresponding to a given user.